# ASSIGNMENT 4: ADAPTIVE NETWORK DESIGN & OPTIMIZATION FRAMEWORK

**Course:** Design and Analysis of Algorithms (DAA) | **Faculty:** Prof. Geeta Tripathi

**Complete Project Overview:** Scenario, Architecture, Conditions, Constraints, Algorithmic Working & Complexity Analysis

## 1. Real-World Scenario & What We Built

A city or enterprise telecommunications operator manages a heterogeneous network of **Data Centers**, **Regional Network Hubs**, and **IoT Gateways**. Because network topology density, telemetry workloads, fault rates, capital budgets, and maintenance site counts change dynamically throughout the day, **a single static algorithm is suboptimal across all operating regimes**.

Our framework solves **6 interconnected network operations** inside one unified, self-tuning workflow:

1. **Minimum-Cost Backbone (Module 1):** Connect all hubs, data centers, and gateways at minimum fiber installation cost without redundant cycles.
2. **All-Pairs Cost Analysis (Module 2):** Compute shortest communication latency paths between every pair of major network zones.
3. **Network-State Data Management (Module 3):** Index high-speed real-time telemetry in RAM and historical state audit logs on persistent disk blocks.
4. **Fault Event Prioritization (Module 4):** Triage active fault alarms and dynamically escalate urgency during cascading failure storms.
5. **Upgrade Budgeting (Module 5):** Allocate a constrained capital budget across divisible bandwidth leases and indivisible hardware units to maximize network utility.
6. **Field Engineer Maintenance Tour (Modules 6 & 7):** Route a field engineer from a central depot through all damaged network sites and back to base using exact TSP search, with NP-Hard scalability safeguards.

> **Unique Selling Points (USPs) of Our Framework:**
>
> - **Runtime Self-Tuning Algorithm Selection:** Inspects mathematical properties of the live input (Graph Density `D`, Storage Tier, Escalation Ratio `R`, Asset Divisibility, and Cluster Size `N`) to automatically switch algorithms at runtime.
> - **End-to-End Connected Pipeline:** Module 2's Floyd-Warshall all-pairs latency matrix directly feeds Module 6's Traveling Salesperson (TSP) dispatch engine to route technicians over actual shortest network corridors.
> - **Built-In Operation Telemetry:** Tracks edge relaxations, tree rotations, simulated disk block reads/writes, cascading cuts, DP states, and pruned search states to empirically verify asymptotic bounds.

## 2. Overall Framework Architecture & 6-Stage Decision Flow

Every request processed by the framework follows the 6-stage pipeline prescribed in the assignment brief:

**1. SCENARIO INPUT**
Graph topology, telemetry stream, fault alarms, budget, or target sites

**2. PROBLEM MODEL**
Entities, mathematical constraints, objective & input size parameters

**3. DAA CHOICE**
Inspects live input properties against mathematical thresholds

**4. EXECUTION**
Runs selected DAA syllabus algorithm or data structure

**5. ANALYSIS**
Records runtime (ms), operation counts & Time/Space bounds

**6. ADAPTATION**
Automatically switches choice when scenario conditions change

### Core Layer ( `core/` )

Models `Node` (Data Center, Hub, IoT Gateway), `NetworkEdge` (cost, latency, bandwidth, reliability), and `NetworkGraph` (adjacency lists, matrix, density `D = 2|E| / (|V|(|V|-1))` ).

### Module 1: Backbone MST ( `algorithms/mst/` )

`AdaptiveMSTSelector` inspects graph density `D` : dispatches **Kruskal's Algorithm** (with Disjoint-Set) when `D < 0.35` (sparse) and switches to **Prim's Algorithm** when `D >= 0.35` (dense).

### Module 2: All-Pairs Routing ( `algorithms/routing/` )

`FloydWarshallRouter` computes complete pairwise shortest paths in `O(|V|³)` time using Dynamic Programming, reconstructs hop-by-hop routes, and detects negative cycles.

### Module 3: State Indexing ( `indexing/` )

`AdaptiveIndexSelector` dispatches live volatile telemetry to an in-memory **Red-Black Tree** ( `<= 2` rotations/insert) and historical logs to an **Order-t B-Tree** ( `t = 16` disk blocks).

### Module 4: Event Prioritization ( `prioritization/` )

`AdaptiveTriageSelector` uses a **Binomial Heap** in steady state and dynamically migrates live alarms to a **Fibonacci Heap** when the `decrease_key` escalation ratio reaches `>= 0.20` .

### Module 5 & 6: Budgeting & TSP Dispatch ( `budgeting/` , `dispatch/` )

`AdaptiveBudgetAllocator` switches between **Fractional Knapsack (Greedy)** and **0/1 Knapsack (DP)**. `AdaptiveDispatchEngine` compares **DFS Backtracking** vs. **Branch-and-Bound TSP**.

## 3. Master Specification: Entities, Constraints, Conditions & Justifications

| Module | Entities & Input Size | Mathematical Constraints | Objective Function | Switching Condition & Threshold | Chosen DAA Technique & Reason |
|---|---|---|---|---|---|
| **1. Minimum-Cost Backbone (MST)** | **Entities:** Hubs, Data Centers, Fiber Links **Size:** `\|V\|` vertices, `\|E\|` edges | 1. Span all `\|V\|` nodes 2. Acyclic connected tree ( `\|V\| - 1` edges) 3. Edge weight `w(e) >= 0` | `Min Sum w(e)` for `e in T` | **Edge Density:** `D = 2\|E\| / (\|V\|(\|V\|-1))` **Threshold:** `D = 0.35` | **Sparse ( `D < 0.35` ): Kruskal's** ( `O(E log E)` ) — fast sorting on few edges + near-constant `O(α(V))` Union-Find. **Dense ( `D >= 0.35` ): Prim's** ( `O(V²)` ) — avoids sorting `O(V²)` edges. |

| Module | Entities & Input Size | Mathematical Constraints | Objective Function | Switching Condition & Threshold | Chosen DAA Technique & Reason |
|---|---|---|---|---|---|
| **2. All-Pairs Cost Analysis** | **Entities:** Network Zones & Transit Links **Size:** `\|V\|` zones | 1. Weighted directed or undirected graph 2. No negative-weight cycles ( `D[i][i] >= 0` ) | `Min Latency(i -> j)` for all pairs `(i, j) in V × V` | **Full Pairwise Matrix Required** for planning & TSP | **Floyd-Warshall DP ( `O(V³)` time, `O(V²)` space):** Computes all pairs in a tight 3-nested matrix loop and detects negative cycles, unlike Dijkstra. |
| **3. Network-State Indexing** | **Entities:** Telemetry & Audit Logs **Size:** `N` records, degree `t = 16` | 1. Keys strictly ordered for range queries 2. Logarithmic height balance | Minimize RAM rotation overhead or Disk Block I/O seeks | **Storage Tier:** `VOLATILE_TELEMETRY` (RAM) vs. `HISTORICAL_LOGS` (Disk) | **RAM Tier: Red-Black Tree** ( $O(\log_2 N)$ ) — guarantees `<= 2` rotations per insert. **Disk Tier: Order-t B-Tree** ( `O(log_t N)` ) — packs `2t-1` keys per block to slash disk I/O seeks. |
| **4. Fault Event Prioritization** | **Entities:** Fault Alarms **Size:** `N` alarms, `K` escalations | 1. Min-Heap order ( `key(parent) <= key(child)` ) 2. Priority score `> 0` | Extract minimum priority score & coalesce regional queues | **Escalation Ratio:** `R = K / (N + K)` **Threshold:** `R = 0.20` | **Steady ( `R < 0.20` ): Binomial Heap** ( `O(log N)` worst-case ops). **Storm ( `R >= 0.20` ): Fibonacci Heap** ( `O(1)` amortized `decrease_key` and `union` via cascading cuts). |
| **5. Upgrade Budgeting** | **Entities:** Bandwidth Leases & Router Hardware **Size:** `N` items, Budget `W` | 1. Total spend `Sum(w_i * x_i) <= W` 2. `w_i > 0, v_i >= 0` | `Max Sum(v_i * x_i)` for `i = 1..N` | **Asset Divisibility:** Continuous `x_i in [0, 1]` vs. Discrete `x_i in {0, 1}` | **Divisible: Fractional Knapsack Greedy** ( `O(N log N)` via `v_i / w_i` ratio). **Indivisible: 0/1 Knapsack DP** ( `O(N * W)` ) — greedy fails on whole hardware units due to budget gaps. |
| **6. Maintenance Tour (TSP)** | **Entities:** Depot + Damaged Sites **Size:** `N` sites, Matrix `C[i][j]` | 1. Start and end at Depot `v_0` 2. Visit every site **exactly once** (Hamiltonian cycle) | `Min Sum C[π(i), π(i+1)] + C[π(N-1), π(0)]` | **Cluster Size `N` : Exact Cutoff:** `N <= 14` | **Small ( `N <= 14` ): Exact Backtracking vs. Exact Branch-and-Bound** (Reduced Cost Matrix LB prunes `>= 85%` of states). **Large ( `N > 14` ): Bounded Heuristic** due to NP-Hard `O((N-1)!)` growth. |

## 4. How Each Algorithm Works Step-by-Step & Why Alternatives Were Rejected

### Module 1: Minimum-Cost Backbone — Kruskal's vs. Prim's Algorithm

- **How Kruskal's Works (Sparse Regime `D < 0.35`):** Sorts all `|E|` links by weight `w(e)` and iterates from cheapest to most expensive. Uses a **Disjoint-Set (Union-Find)** with **Path Compression** and **Union by Rank** to check in `O(α(V))` time whether endpoints `(u, v)` are already connected. Adds `|V| - 1` acyclic links.
- **How Prim's Works (Dense Regime `D >= 0.35`):** Grows a single spanning tree from the Data Center root by repeatedly attaching the cheapest edge crossing from the tree to an unvisited hub. For dense graphs, the `O(|V|²)` matrix scan avoids sorting `O(|V|²)` edges.
- **Why Alternatives Were Rejected:** BFS/DFS ignore edge weights; Kruskal's `O(E log E)` sorting bottlenecks when `E ≈ V²`.

### Module 2: All-Pairs Cost Analysis — Floyd-Warshall Dynamic Programming

- **How It Works:** Initializes distance matrix `D[i][j]` from direct link latencies and evaluates the DP recurrence `D^(k)[i][j] = min(D^(k-1)[i][j], D^(k-1)[i][k] + D^(k-1)[k][j])` across all intermediate hubs `k = 0..|V|-1` while maintaining a `NextHop[i][j]` matrix for route reconstruction. Checks `D[i][i] < 0` to detect negative cycles.
- **Why Alternatives Were Rejected: Dijkstra's Algorithm** is Single-Source, fails on negative weights, and running it `|V|` times on dense graphs takes `O(V³ log V)` with heavy heap overhead. Repeated **Bellman-Ford** takes `O(V⁴)` on dense graphs.

### Module 3: Network-State Data Management — Red-Black Tree vs. Order-t B-Tree

- **How Red-Black Tree Works (RAM Tier):** Enforces 5 color invariants (black root, no adjacent red nodes, uniform black-height). Insertions restore balance using recoloring and **at most 2 rotations**, providing fast, predictable $O(\log_2 N)$ in-memory performance.
- **How Order-t B-Tree Works (Disk Tier):** Packs between `t - 1` and `2t - 1` sorted keys into a single disk block (`t = 16` stores up to 31 keys/node), splitting full nodes in a single downward pass. Compresses tree height to `O(log_t N)` (2–3 levels), minimizing slow physical disk seeks.
- **Why Alternatives Were Rejected: Hash Tables** scramble key order and cannot perform time-window **range queries**; **AVL Trees** require up to `O(log N)` rotations per write.

### Module 4: Fault Event Prioritization — Binomial Heap vs. Fibonacci Heap

- **How Binomial Heap Works (Steady State):** Stores alarms in a forest of Binomial Trees `B_k` (at most one tree per degree), merging trees of equal degree in worst-case `O(log N)` time for `insert`, `union`, `extract_min`, and `decrease_key`.
- **How Fibonacci Heap Works (Escalation Storm):** Maintains a lazy circular doubly-linked root list where `insert` and `union` take **`O(1)` time**. When an alarm escalates (`decrease_key`), it cuts the violated node into the root list in **`O(1)` amortized time** and uses **Cascading Cuts** on marked ancestors, deferring tree consolidation until `extract_min`.
- **Why Alternatives Were Rejected:** Standard **Binary Heaps** require `O(N)` linear time to merge two regional queues and `O(log N)` for every `decrease_key` escalation.

### Module 5: Upgrade Budgeting — Fractional Knapsack (Greedy) vs. 0/1 Knapsack (DP)

- **How Fractional Knapsack Works (Divisible Bandwidth):** Sorts bandwidth leases by value-to-cost density `v_i / w_i` in `O(N log N)` time, funding top items whole and taking an exact fractional slice of the boundary item.
- **How 0/1 Knapsack Works (Indivisible Hardware):** Evaluates DP table `DP[i][w] = max(DP[i-1][w], v_i + DP[i-1][w - w_i])` across `N` items and budget capacities `w = 0..W`, then backtracks from `DP[N][W]` to select whole hardware units.
- **Why Alternatives Were Rejected:** Greedy fails on 0/1 items because it leaves unspent budget fragments; Brute-Force subset enumeration takes `O(2^N)` exponential time.

### Modules 6 & 7: Maintenance Tour (TSP) — DFS Backtracking vs. Branch-and-Bound

- **How DFS Backtracking Works (`dispatch/backtracking.py`):** Recursively explores permutations of unvisited sites depth-first, pruning a branch only when the partial path cost already exceeds the best complete tour found so far.
- **How Branch-and-Bound Works (`dispatch/branch_and_bound.py`):** Initializes a tight Upper Bound (`UB`) via Nearest Neighbor + 2-Opt, and computes a **Reduced Cost Matrix Lower Bound (`LB`)** at every node by subtracting row and column minimums. Uses a Min-Heap (Best-First Search) to expand the lowest-`LB` node first and immediately prunes entire subtrees whenever `LB >= UB`. On 8 sites, this reduces explored states from **6,171 (Backtracking) down to 45 (Branch-and-Bound)** while guaranteeing the exact same optimal tour!

> **5. Mandatory Complexity Classifications (From Page 2 Warning Box of PDF):**
>
> - **Optimization TSP vs. Decision TSP:** *Optimization TSP* ("Find the closed tour visiting all sites with minimum total cost") is **NP-Hard**. *Decision TSP* ("Does there exist a closed tour with total cost <= K?") is **NP-Complete** (verifiable in `O(V)` time and reducible from Hamiltonian Cycle).
> - **Why 0/1 Knapsack DP is Pseudo-Polynomial (Not Strictly Polynomial):** The DP algorithm runs in `O(N * W)` time. Because the numeric budget value `W` takes `b = log₂(W)` bits to encode in binary, substituting `W = 2^b` yields `O(N * 2^b)`, which is **exponential in the input bit-length `b`** — hence **Pseudo-Polynomial**. Its decision version is **NP-Complete**, whereas Fractional Knapsack (`O(N log N)`) is strictly **Polynomial**.

## 6. Gaps Identified in Classic DAA Algorithms That Enabled Modern Algorithms

| Classic Syllabus Technique | Identified Limitation / Scalability Gap | Modern Algorithm Enabled by This Gap |
|---|---|---|
| **Prim's & Kruskal's MST** | Sequential single-threaded bottleneck on billion-edge graphs. | **Filter-Kruskal** & **Parallel Distributed Boruvka** |
| **Floyd-Warshall ( $\Theta(V^3)$ )** | Performs cubic work even when the network is very sparse ( $E << V^2$ ). | **Johnson's Algorithm ( $O(V^2 \log V + VE)$ )** & **Contraction Hierarchies** |
| **Order-t B-Tree** | Random in-place block updates cause high write amplification on SSDs. | **LSM-Trees (Log-Structured Merge Trees)** in RocksDB & Cassandra |
| **Fibonacci Heap** | 4 pointers per node cause poor CPU cache locality and high constant overhead. | **Pairing Heaps** & **Rank-Pairing Heaps** |
| **0/1 Knapsack DP ( $O(N*W)$ )** | Table memory explodes when budget $W$ is huge with fine precision. | **FPTAS (Fully Polynomial-Time Approximation Scheme)** via value scaling |
| **Exact TSP B&B / Backtracking** | Factorial state space explosion when site count exceeds $N > 25$ . | **Christofides' 1.5-Approximation** & **Lin-Kernighan-Helsgaun (LKH)** |